DEVOPS
& CLOUD GRIMOIRE

LINUX COMMAND GRIMOIRE | VOL. 3 | ANIME CHEAT-SHEET EDITION



- CREATED BY -
RUTUJA NAIK

30 CHAPTERS * 493 ENTRIES * 7 CODE SCROLLS

01 SURVIVAL HABITS

Rules every adventurer learns

`cp file file.bak`
back up before you edit.

`ls *.tmp`
check the wildcard before you rm it.

`echo rm *.log`
dry run: see what the glob expands to.

`sudo !!`
re-run the last command as root.

`!$`
reuse the last argument of the previous command.

`man -k keyword`
search manual pages by keyword.

`cmd --help | less`
read usage without losing your place.

`tldr tar`
short practical examples (if installed).

`history | grep cmd`
find that command you ran last week.

`script session.log`
record your terminal session to a file.

`type -a cmd`
show every place a command comes from.

`alias k=kubectl`
short aliases save thousands of keystrokes.

`--dry-run`
many tools can preview: rsync, apt, kubectl.

`pwd; ls`
know where you are before you delete.

SENPAI SAYS
Pause before pressing Enter on anything with rm, dd or a wildcard.

02 BASH SCRIPTING BASICS

Writing your first spellbook

`#!/usr/bin/env bash`
shebang: picks the interpreter.

`chmod +x run.sh && ./run.sh`
make it executable, then run it.

`set -euo pipefail`
stop on error, unset var or failed pipe.

`name="Sam"`
assign a variable (no spaces around =).

`${VAR:-default}`
use a default when VAR is empty.

`$1 $2 $@ $#`
args: first, second, all, and count.

`read -p "Name: " n`
ask the user for input.

`if [[ -f file ]]; then ...; fi`
test whether a file exists.

`[[ $a -gt 5 ]]`
numeric tests: -eq -ne -gt -lt -ge -le.

`for f in *.txt; do ...; done`
loop over files.

`while read -r l; do ...; done < f`
read a file line by line.

`fn() { echo "hi"; }`
define a function.

`$(command)`
capture command output as a value.

`trap 'rm -f /tmp/x' EXIT`
clean up when the script exits.

`bash -n script.sh`
check syntax without running.

`bash -x script.sh`
debug: print each command as it runs.

`shellcheck script.sh`
lint your script for common bugs.

`source file.sh`
run a script in the current shell.

SPELL SCROLL | hello.sh

```
#!/usr/bin/env bash
set -euo pipefail

name="${1:-world}"
echo "Hello, $name!"

for i in 1 2 3; do
  echo "Round $i"
done
```

SENPAI SAYS
Quote your variables: "$var". Unquoted ones break on spaces.

03 EXIT CODES & SIGNALS

Reading the omens

`echo $?`
exit code of the last command.

`exit 0`
success.

`exit 1`
general error.

`exit 2`
misuse of a shell builtin or command.

`exit 126`
found but cannot execute (permissions).

`exit 127`
command not found.

`exit 130`
stopped with Ctrl+C (128 + 2).

`exit 137`
killed by SIGKILL (128 + 9). Often OOM.

`exit 143`
ended by SIGTERM (128 + 15).

`kill -l`
list signal names and numbers.

`kill -15 PID`
SIGTERM: ask nicely to stop (default).

`kill -HUP PID`
SIGHUP: many daemons reload config.

`kill -STOP PID`
pause a process (-CONT resumes it).

`kill -9 PID` DANGER
SIGKILL: cannot be caught or ignored.

`timeout 10 cmd`
stop a command after 10 seconds.

`cmd1 && cmd2 || cmd3`
if / else in one line.

SENPAI SAYS
Exit code 0 means success. Anything else means something went wrong.

04 REGEX & TEXT WIZARDRY
Pattern magic

`grep -E "err|warn" file`
match either word (extended regex).

`grep -o "[0-9]\+" file`
print only the matching part.

`grep -w "root" file`
match whole words only.

`grep -A3 -B3 "err" file`
show 3 lines after and before a match.

`grep -l "text" *.conf`
list only the filenames that match.

`^ $ . *`
regex: start, end, any char, repeat.

`[abc] [^abc] \d`
regex: set, not-in-set, digit (grep -P).

`awk -F: '{print $1,$3}' /etc/passwd`
print each user and UID.

`awk '{s+=$1} END {print s}' f`
sum the first column.

`awk 'NR>1' file`
skip the header line.

`sed -n '5,10p' file`
print only lines 5 to 10.

`sed '/^#/d' file`
delete comment lines.

`sed -E 's/(a) (b)/\2 \1/'`
swap two words using groups.

`sort -k2 -n file`
sort by 2nd column, numerically.

`sort -u file`
sort and remove duplicates.

`comm -12 a.txt b.txt`
lines common to two sorted files.

`tr -s ' '`
squeeze repeated spaces into one.

`fold -w 80 file`
wrap long lines at 80 characters.

`column -s, -t file.csv`
view a CSV as a tidy table.

`xargs -n1 cmd`
run cmd once per input item.

SENPAI SAYS
Learn grep, sed and awk well and you can skip a lot of scripting.

05 SERVER SLOW? 60S CHECK
The first-minute triage

`uptime`
1. load averages for 1, 5 and 15 minutes.

`dmesg -T | tail`
2. kernel errors: OOM kills, disk faults.

`vmstat 1 5`
3. CPU, memory, swap and IO each second.

`mpstat -P ALL 1`
4. per-CPU usage. Is one core pinned?

`pidstat 1`
5. which process is eating CPU.

`iostat -xz 1`
6. disk latency and utilization.

`free -m`
7. memory used, cached and swap.

`sar -n DEV 1`
8. network throughput per interface.

`sar -n TCP,ETCP 1`
9. TCP connections and retransmits.

`top`
10. confirm the suspect interactively.

`nproc`
how many CPUs? compare with load.

`ps aux --sort=-%cpu | head`
top CPU users.

`ps aux --sort=-%mem | head`
top memory users.

`journalctl -p err -b`
errors since the last boot.

`dmesg -T | grep -i oom`
find out-of-memory kills.

SENPAI SAYS
load > nproc means tasks are waiting. sar/iostat/mpstat come from sysstat.

06 DISK FULL RESCUE
Clearing the vault

`df -h`
1. which filesystem is full?

`df -i`
2. out of inodes? (too many tiny files)

`du -xh / --max-depth=1 | sort -rh | head`
3. biggest top-level folders.

`du -ah /var | sort -rh | head -20`
20 biggest items under /var.

`find / -xdev -size +500M`
4. files over 500 MB on this disk.

`lsof +L1`
5. deleted files still held open.

`journalctl --disk-usage`
how big is the systemd journal?

`journalctl --vacuum-size=200M`
shrink the journal to 200 MB.

`apt clean`
clear the apt download cache.

`docker system df`
how much space Docker uses.

`docker system prune -a` DANGER
remove unused Docker data.

`truncate -s 0 big.log` DANGER
empty a log without deleting it.

`logrotate -f /etc/logrotate.conf`
force log rotation now.

`ncdu /`
interactive hunt for space hogs.

SENPAI SAYS
Deleted a big file but df is still full? A process holds it open. Restart it.

07 CAN'T CONNECT? DEBUG
Following the signal

`ping -c 3 8.8.8.8`
1. is the internet reachable by IP?

`ping -c 3 example.com`
2. does name resolution work?

`dig +short example.com`
3. which IP does DNS return?

`ip route get 8.8.8.8`
4. route and source IP in use.

`nc -zv host 443`
5. is the TCP port open?

`curl -v https://example.com`
6. full request and response detail.

`curl -o /dev/null -s -w "%{http_code}\n" url`
print just the HTTP status.

`ss -tulpn | grep :8080`
is anything listening locally?

`sudo iptables -S`
7. any firewall rule blocking it?

`sudo ufw status numbered`
UFW rules with numbers.

`traceroute -n host`
where does the path stop?

`mtr -rwc 10 host`
10-packet path report with loss %.

`openssl s_client -connect host:443 -servername host`
inspect the TLS handshake.

`tcpdump -nn -i any port 53`
watch DNS traffic live.

`resolvectl status`
which DNS servers are in use.

`curl --resolve host:443:1.2.3.4 https://host`
test a server before DNS changes.

SENPAI SAYS
Work bottom-up: link, IP, DNS, port, firewall, then the app itself.

08 PROCESS DEEP DIVE
Peering into the engine

`strace -p PID`
watch system calls of a live process.

`strace -c cmd`
summary of system calls and time.

`strace -f -e openat cmd`
follow forks, show files opened.

`ltrace cmd`
trace library calls.

`lsof -p PID`
files and sockets a process has open.

`ls -l /proc/PID/fd`
open file descriptors via /proc.

`cat /proc/PID/status`
state, memory and threads for a PID.

`cat /proc/meminfo`
detailed memory breakdown.

`pmap -x PID`
memory map of a process.

`perf top`
live view of the hottest functions.

`iotop -o`
processes doing disk IO (root).

`pidstat -d 1`
per-process disk read and write.

`ulimit -n`
max open files for this shell.

`sysctl vm.swappiness`
read a kernel setting.

`sudo sysctl -w vm.swappiness=10`
change it until reboot.

`taskset -c 0,1 cmd`
pin a command to CPUs 0 and 1.

`ps -eo pid,ppid,stat,cmd | grep ' Z'`
find zombie processes.

SENPAI SAYS
strace shows what a stuck program is actually doing. Great for mysteries.

09 LVM & STORAGE
Resizing the vault

`pvcreate /dev/sdb`
prepare a disk for LVM.

`vgcreate vg0 /dev/sdb`
create a volume group.

`lvcreate -L 20G -n data vg0`
create a 20 GB logical volume.

`pvs vgs lvs`
summaries of PVs, VGs and LVs.

`lvextend -L +10G /dev/vg0/data`
grow the volume by 10 GB.

`resize2fs /dev/vg0/data`
grow an ext4 filesystem to fit.

`xfs_growfs /mount/point`
grow an XFS filesystem.

`lvextend -r -L +10G /dev/vg0/data`
grow volume and filesystem together.

`fallocate -l 2G /swapfile`
create a 2 GB swap file.

`chmod 600 /swapfile && mkswap /swapfile`
secure it, then format as swap.

`swapon /swapfile`
enable the swap file.

`swapon --show`
list active swap.

`df -hT`
usage plus filesystem types.

`mount -o remount,rw /`
remount root read-write.

`tune2fs -l /dev/sdX1`
show ext4 filesystem details.

`cat /proc/mdstat`
quick software RAID health.

`mdadm --detail /dev/md0`
RAID array status.

`smartctl -H /dev/sda`
disk health: PASSED or FAILED.

SENPAI SAYS
Order matters: extend the logical volume first, then the filesystem.

10 SYSTEMD POWER USER
Mastering the summoner

`systemctl cat nginx`
show the full unit file.

`sudo systemctl edit nginx`
override settings safely.

`systemctl list-units --type=service`
all loaded services.

`systemctl is-enabled nginx`
starts at boot?

`systemctl is-active nginx`
running right now?

`systemctl mask nginx`
block a service from starting at all.

`systemctl get-default`
current boot target.

`systemctl set-default multi-user.target`
boot to text mode.

`systemctl show -p MainPID nginx`
get the main PID.

`journalctl -u app -f`
follow one service live.

`journalctl -b -1`
logs from the previous boot.

`journalctl --vacuum-time=7d`
keep only 7 days of logs.

`systemd-analyze critical-chain`
what delays boot.

`loginctl list-sessions`
who is logged in via systemd.

SPELL SCROLL | /etc/systemd/system/app.service

```
[Unit]
Description=My App
After=network.target

[Service]
User=app
ExecStart=/usr/bin/python3 /opt/app/main.py
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

SENPAI SAYS
After editing any unit file, run systemctl daemon-reload.

11 CRON & SCHEDULING PRO
Timekeeper's charms

`* * * * *`
every minute.

`*/5 * * * *`
every 5 minutes.

`0 * * * *`
every hour, on the hour.

`0 2 * * *`
every day at 2:00 AM.

`0 9 * * 1-5`
weekdays at 9:00 AM.

`*/15 9-17 * * 1-5`
every 15 min during work hours.

`0 0 1 * *`
first day of every month.

`@reboot`
run once at startup.

`flock -n /tmp/job.lock job.sh`
stop overlapping runs of a job.

`crontab -l | grep -v '^#'`
show only active cron jobs.

`systemctl list-timers --all`
systemd timers and next run.

`grep CRON /var/log/syslog`
did my cron job actually run?

SENPAI SAYS
Fields: minute hour day-of-month month weekday. Use full paths in cron jobs.

12 LOGS & MONITORING

Watching the horizon

`tail -f /var/log/syslog`
Debian/Ubuntu system log, live.

`tail -f /var/log/messages`
RHEL-family system log, live.

`sudo less /var/log/auth.log`
logins and sudo (RHEL: /var/log/secure).

`lastb`
failed login attempts.

`last -n 10`
last 10 logins.

`zgrep "ERROR" app.log.gz`
search compressed logs.

`awk '{print $9}' access.log | sort | uniq -c | sort -rn`
count HTTP status codes.

`awk '{print $1}' access.log | sort | uniq -c | sort -rn | head`
top client IPs.

`logger "deploy started"`
write a custom line to syslog.

`logrotate -d /etc/logrotate.conf`
dry-run log rotation.

`watch -n 5 'ss -s'`
socket summary every 5 seconds.

`ps aux | grep [n]ginx`
find a process without matching grep.

`curl -s localhost:9100/metrics | head`
peek at node_exporter metrics.

`promtool check config prometheus.yml`
validate a Prometheus config.

`systemctl status node_exporter`
is the metrics agent running?

SENPAI SAYS
The pgrep trick: grep for [n]ginx so grep doesn't match itself.

13 WEB SERVER & TLS

Gatekeeper's toolkit

`sudo nginx -t`
test nginx config before reloading.

`sudo nginx -s reload`
reload nginx without downtime.

`nginx -T | less`
dump the full effective config.

`sudo apachectl configtest`
test Apache config.

`tail -f /var/log/nginx/error.log`
watch nginx errors live.

`curl -I -L https://site`
headers, following redirects.

`curl -w "%{time_total}\n" -o /dev/null -s url`
total request time.

`ab -n 100 -c 10 url`
quick load test (apache bench).

`openssl x509 -in cert.pem -noout -dates`
certificate start and expiry.

`echo | openssl s_client -connect host:443 2>/dev/null | openssl x509 -noout -dates`
expiry of a live site's cert.

`openssl req -new -newkey rsa:2048 -nodes -keyout k.pem -out c.csr`
create a key and CSR.

`sudo certbot --nginx -d example.com`
get a Let's Encrypt certificate.

`sudo certbot renew --dry-run`
test automatic renewal.

`sudo certbot certificates`
list certificates and expiry.

`htpasswd -c .htpasswd user`
create a basic-auth password file.

SENPAI SAYS
Always run nginx -t before reload. A typo should not take your site down.

14 SECURITY HARDENING

Forging the shield

`sudo sshd -t`
validate sshd config before restarting.

`PermitRootLogin no`
sshd_config: block direct root login.

`PasswordAuthentication no`
sshd_config: allow keys only.

`passwd -l user`
lock a user account.

`chage -l user`
password aging info.

`last -a | head`
recent logins with source IPs.

`find / -perm -4000 -type f 2>/dev/null`
list SUID binaries.

`find / -xdev -type f -perm -0002`
world-writable files.

`chattr +i file`
make a file immutable.

`lsattr file`
view file attributes.

`sudo ufw default deny incoming`
default-deny firewall policy.

`sudo fail2ban-client set sshd unbanip IP`
unban an IP address.

`getenforce`
SELinux mode: Enforcing or Permissive.

`sestatus`
SELinux full status.

`sudo setenforce 0` DANGER
SELinux permissive until reboot.

`sudo ausearch -m avc -ts recent`
recent SELinux denials.

`sudo aa-status`
AppArmor profile status.

`sudo lynis audit system`
run a security audit.

`clamscan -r /home`
scan for malware.

SENPAI SAYS
Keys only, no root login, default-deny firewall. That alone stops most attacks.

15 GIT FOR DEVOPS
Time-travel, advanced

`git log --oneline -10`
last 10 commits, compact.

`git log -p file`
history of changes to one file.

`git blame file`
who changed each line.

`git show sha`
see one commit's changes.

`git diff --staged`
review what you're about to commit.

`git commit --amend`
edit the last commit.

`git rebase -i HEAD~3`
tidy the last 3 commits.

`git cherry-pick sha`
copy one commit onto this branch.

`git bisect start`
binary-search for the commit that broke it.

`git revert sha`
safe undo: a new commit that reverses one.

`git reflog`
find 'lost' commits.

`git tag -a v1.0 -m "release"`
create an annotated release tag.

`git remote -v`
show remote URLs.

`git fetch --prune`
update refs, drop deleted remote branches.

`git stash pop`
bring your stash back.

`git config --global alias.st status`
create a short alias.

`git worktree add ../fix branch`
second working copy of a repo.

`git clean -fd` DANGER
delete untracked files and folders.

`git push --force-with-lease` DANGER
safer force push.

SENPAI SAYS
git reflog is your safety net. Almost nothing is truly lost.

16 DOCKER ADVANCED
Container mastery

`docker build --no-cache -t app .`
rebuild without the cache.

`docker history image`
see the layers of an image.

`docker inspect container`
full JSON details.

`docker cp c:/path/file .`
copy a file out of a container.

`docker run -v $(pwd):/app image`
mount the current folder inside.

`docker run -e KEY=val image`
pass an environment variable.

`docker run --memory 512m --cpus 1 image`
set resource limits.

`docker logs --tail 100 -f c`
last 100 lines, then follow.

`docker exec c env`
check a container's environment.

`docker top c`
processes inside a container.

`docker network ls`
list networks.

`docker compose config`
validate and print the final compose file.

`docker compose ps`
stack status.

`docker compose logs -f svc`
follow one service.

`docker tag app user/app:1.0`
name an image for a registry.

`docker login`
authenticate to a registry.

`docker push user/app:1.0`
upload the image.

`docker buildx build --platform linux/amd64,linux/arm64 .`
multi-architecture build.

`docker save -o app.tar app`
export an image to a tar file.

`docker load -i app.tar`
import it elsewhere.

`docker volume prune` DANGER
delete unused volumes.

`docker image prune -a` DANGER
delete all unused images.

SPELL SCROLL | Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
USER 1000
EXPOSE 8000
CMD ["python", "main.py"]
```

SENPAI SAYS
Small images: use slim bases, a .dockerignore, and run as a non-root USER.

17 KUBERNETES ADVANCED
Commanding the fleet

`kubectl get pods -o wide`
show node and pod IP.

`kubectl get pods -l app=web`
filter by label.

`kubectl get pods --field-selector=status.phase!=Running`
find unhealthy pods.

`kubectl get events --sort-by=.lastTimestamp`
recent cluster events.

`kubectl describe node name`
node capacity, pods and conditions.

`kubectl logs pod --previous`
logs from the crashed container.

`kubectl rollout status deploy/web`
wait for a rollout.

`kubectl rollout undo deploy/web`
roll back to the previous version.

`kubectl set image deploy/web web=nginx:1.27`
update the image.

`kubectl create deployment web --image=nginx`
quick deployment.

`kubectl expose deployment web --port=80`
create a Service.

`kubectl run tmp --rm -it --image=busybox -- sh`
throwaway debug pod.

`kubectl cp pod:/path ./local`
copy a file out of a pod.

`kubectl get secret s -o jsonpath='{.data.key}' | base64 -d`
decode a secret value.

`kubectl auth can-i create pods`
check your own permissions.

`kubectl diff -f app.yaml`
preview a change before applying.

`kubectl apply --dry-run=client -f app.yaml`
validate without changing anything.

`kubectl explain pod.spec.containers`
built-in field documentation.

`kubectl config use-context name`
switch cluster.

`kubectl config set-context --current --namespace=dev`
set the default namespace.

`kubectl top nodes`
node CPU and memory.

`kubectl cordon node`
mark a node unschedulable.

`kubectl drain node --ignore-daemonsets` DANGER
evict pods before maintenance.

`kubectl delete -f app.yaml` DANGER
delete what a file created.

SPELL SCROLL | deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels: {app: web}
  template:
    metadata:
      labels: {app: web}
    spec:
      containers:
        - name: web
          image: nginx:1.27
```

SENPAI SAYS
CrashLoopBackOff? describe pod, then logs --previous. Exit code 137 = OOMKilled.

18 HELM & KUSTOMIZE
Packaging spells

`helm repo update`
refresh chart lists.

`helm search repo nginx`
find charts.

`helm show values chart`
see configurable values.

`helm install r chart -n ns --create-namespace`
install into a new namespace.

`helm upgrade --install r chart -f values.yaml`
install or upgrade in one command.

`helm template r chart`
render the YAML locally.

`helm lint ./chart`
check a chart for problems.

`helm diff upgrade r chart`
preview changes (diff plugin).

`helm history r`
revision history.

`helm rollback r 1`
go back to revision 1.

`helm get values r`
values currently applied.

`helm uninstall r` DANGER
remove a release.

`kubectl kustomize overlays/dev`
render a kustomize overlay.

`kubectl apply -k overlays/dev`
apply a kustomize overlay.

SENPAI SAYS
helm upgrade --install is idempotent: the same command works first time and later.

19 TERRAFORM

Infrastructure as code

- `terraform init` — download providers, set up the backend.
- `terraform fmt -recursive` — auto-format all .tf files.
- `terraform validate` — check syntax and consistency.
- `terraform plan -out=tfplan` — save the plan to a file.
- `terraform apply tfplan` — apply exactly that saved plan.
- `terraform output` — show output values.
- `terraform state list` — resources Terraform manages.
- `terraform state show addr` — details for one resource.
- `terraform workspace new dev` — create a workspace.
- `terraform workspace list` — list workspaces.
- `terraform import addr id` — adopt an existing resource.
- `terraform init -upgrade` — upgrade provider versions.
- `terraform console` — try expressions interactively.
- `TF_LOG=DEBUG terraform plan` — verbose debug logging.
- `terraform apply -auto-approve` DANGER — apply with no confirmation.
- `terraform destroy` DANGER — tear down everything.
- `terraform force-unlock ID` DANGER — clear a stuck state lock.

SPELL SCROLL | main.tf

```
terraform {
  required_providers {
    aws = { source = "hashicorp/aws" }
  }
}
provider "aws" { region = "ap-south-1" }
resource "aws_s3_bucket" "demo" {
  bucket = "my-unique-bucket-name"
}
```

SENPAI SAYS

Read every plan. Look for destroy and replace lines before you apply.

20 ANSIBLE

Automation familiars

- `ansible all -m ping -i inventory` — test connection to all hosts.
- `ansible web -a "uptime"` — run an ad-hoc command.
- `ansible-playbook site.yml` — run a playbook.
- `ansible-playbook site.yml --check --diff` — dry run and show changes.
- `ansible-playbook site.yml -l web` — limit to one host group.
- `ansible-playbook site.yml -e "ver=2"` — pass an extra variable.
- `ansible-playbook site.yml --tags deploy` — run only tagged tasks.
- `ansible-playbook site.yml -vvv` — maximum verbosity.
- `ansible-inventory --list` — show the parsed inventory.
- `ansible-doc apt` — documentation for a module.
- `ansible-vault encrypt secrets.yml` — encrypt a file.
- `ansible-vault edit secrets.yml` — edit an encrypted file.
- `ansible-galaxy install geerlingguy.nginx` — install a community role.
- `ansible-lint site.yml` — lint your playbook.

SPELL SCROLL | site.yml

```
- hosts: web
  become: true
  tasks:
    - name: Install nginx
      ansible.builtin.apt:
        name: nginx
        state: present
    - name: Start nginx
      ansible.builtin.service:
        name: nginx
        state: started
```

SENPAI SAYS

Idempotent means safe to re-run: the second run should change nothing.

21 CI/CD & AUTOMATION

The deployment pipeline

- `gh auth login` — log in to GitHub from the terminal.
- `gh repo clone owner/repo` — clone a repository.
- `gh pr create --fill` — open a PR using commit info.
- `gh pr list` — list open pull requests.

`gh pr checkout 42`
check out PR number 42.

`gh run list`
recent workflow runs.

`gh run watch`
follow a running workflow live.

`gh workflow run ci.yml`
trigger a workflow manually.

`gh release create v1.0 --generate-notes`
publish a release.

`act`
run GitHub Actions locally (if installed).

`pre-commit run --all-files`
run all pre-commit hooks.

`make build`
run a Makefile target.

`envsubst < tpl.yaml > out.yaml`
fill ${VARS} in a template.

`jq -r '.items[].metadata.name'`
extract raw names from JSON.

`yq '.spec.replicas' file.yaml`
read a YAML value.

`curl -X POST -H "Authorization: Bearer $TOKEN" -d @body.json url`
call an API with a token.

`openssl rand -base64 32`
generate a random secret.

SPELL SCROLL | .github/workflows/ci.yml

```yaml
name: ci
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: make test
```

SENPAI SAYS
Small automated steps beat big manual ones. Make CI run your tests on every push.

22 AWS CLI PRO
Sky kingdom: Amazon

`aws configure --profile dev`
set up a named profile.

`export AWS_PROFILE=dev`
use that profile in this shell.

`aws configure list`
show where credentials come from.

`aws s3 ls s3://bucket --recursive --human-readable`
list objects with sizes.

`aws s3 presign s3://bucket/key --expires-in 3600`
temporary download link (1 hour).

`aws s3 sync dir s3://bucket/ --delete` DANGER
mirror a folder to S3.

`aws s3 rm s3://bucket/ --recursive` DANGER
delete everything under a path.

`aws ec2 describe-instances --query 'Reservations[].Instances[].InstanceId' --output text`
list instance IDs only.

`aws ec2 start-instances --instance-ids i-123`
start an instance.

`aws ec2 stop-instances --instance-ids i-123`
stop an instance.

`aws ec2 describe-security-groups`
list security groups.

`aws iam list-users`
list IAM users.

`aws ssm start-session --target i-123`
shell into EC2 without SSH keys.

`aws logs tail /aws/lambda/fn --follow`
live CloudWatch logs.

`aws lambda invoke --function-name fn out.json`
invoke a Lambda function.

`aws eks update-kubeconfig --name cluster`
connect kubectl to EKS.

`aws ecr get-login-password | docker login --username AWS --password-stdin ACCT.dkr.ecr.REGION.amazonaws.com`
log Docker in to ECR.

`aws secretsmanager get-secret-value --secret-id name`
read a secret.

`aws cloudformation describe-stacks`
list CloudFormation stacks.

SENPAI SAYS
Use --query (JMESPath) and --output table to shape results without jq.

23 AZURE CLI
Sky kingdom: Microsoft

`az login --use-device-code`
log in from a headless machine.

`az account list -o table`
list subscriptions.

`az account set -s SUB`
choose a subscription.

`az configure --defaults group=rg`
set a default resource group.

`az group create -n rg -l eastus`
create a resource group.

`az group list -o table`
list resource groups.

`az resource list -g rg -o table`
everything in a group.

`az vm list -g rg -o table`
VMs in a group.

`az vm start -g rg -n vm`
start a VM.

`az vm deallocate -g rg -n vm`
stop a VM and stop compute billing.

`az aks get-credentials -g rg -n cluster`
connect kubectl to AKS.

`az acr login -n registry`
log Docker in to ACR.

`az storage account list -o table`
list storage accounts.

`az network nsg list -o table`
list network security groups.

`az monitor activity-log list --max-events 10`
recent activity.

`az group delete -n rg` DANGER
delete a group and all its resources.

SENPAI SAYS
Groups hold everything. Deleting a resource group deletes all inside it.

24 GOOGLE CLOUD CLI
Sky kingdom: Google

`gcloud config set project ID`
choose the active project.

`gcloud config list`
show the active config.

`gcloud projects list`
list projects.

`gcloud auth application-default login`
credentials for local SDK code.

`gcloud compute ssh vm --zone ZONE`
SSH into a VM.

`gcloud compute instances stop vm --zone ZONE`
stop a VM.

`gcloud container clusters get-credentials NAME --region R`
connect kubectl to GKE.

`gcloud storage ls`
list buckets.

`gcloud storage cp file gs://bucket/`
upload a file.

`gcloud iam service-accounts list`
list service accounts.

`gcloud run deploy svc --image IMG --region R`
deploy to Cloud Run.

`gcloud logging read "severity>=ERROR" --limit 10`
recent error logs.

`gcloud services enable compute.googleapis.com`
enable an API.

`gcloud components update`
update the gcloud CLI.

`gcloud projects delete ID` DANGER
schedule project deletion.

SENPAI SAYS
Run gcloud config list first so you know which project you are about to change.

25 CLOUD NETWORKING & DNS
Bridges between realms

`ip -s link`
per-interface packet and error counters.

`ip neigh`
ARP / neighbour table.

`ethtool eth0`
NIC speed, duplex and link state.

`dig +trace example.com`
follow DNS from the root servers.

`dig MX example.com`
mail records.

`dig -x 8.8.8.8`
reverse DNS lookup.

`dig @8.8.8.8 example.com`
ask a specific DNS server.

`whois example.com`
domain registration info.

`host example.com`
quick DNS lookup.

`sudo iptables -t nat -L -n -v`
NAT rules with counters.

`sudo nft list ruleset`
show nftables rules.

`sysctl net.ipv4.ip_forward`
is IP forwarding on?

`sudo tcpdump -nn -i any host 10.0.0.5`
capture traffic for one host.

`sudo tcpdump -i eth0 -w cap.pcap`
save a capture for Wireshark.

`ipcalc 10.0.0.0/24`
network, mask, broadcast, host range.

`ss -ant state established`
established TCP connections.

`nmap -p 1-1000 host`
scan a port range.

`curl ifconfig.me`
show your public IP.

`curl -s http://169.254.169.254/latest/meta-data/`
AWS metadata (IMDSv1; v2 needs a token).

SENPAI SAYS
AWS reserves 5 IPs per subnet, so a /28 gives only 11 usable addresses.

26 PORTS TO REMEMBER
Gate numbers

22/tcp
SSH and SFTP.

25/tcp
SMTP (mail sending).

53/udp+tcp
DNS.

80/tcp
HTTP.

123/udp
NTP time sync.

389 / 636
LDAP / LDAPS.

443/tcp
HTTPS.

2375 / 2376
Docker daemon (2376 uses TLS).

2379 / 2380
etcd client / peer.

3000
Grafana (default).

3306
MySQL / MariaDB.

3389
Windows RDP.

5432
PostgreSQL.

6379
Redis.

6443
Kubernetes API server.

8080
common alternate HTTP, Jenkins.

9090
Prometheus.

9100
node_exporter.

9200
Elasticsearch.

10250
kubelet API.

27017
MongoDB.

SENPAI SAYS
Ports below 1024 need root to bind. Know these by heart for interviews.

27 HTTP STATUS CODES

Messages from the server

200
OK.

201
Created.

204
No content (success, empty body).

301
Moved permanently.

302
Found (temporary redirect).

304
Not modified (use cache).

400
Bad request.

401
Unauthorized: missing or bad login.

403
Forbidden: logged in, not allowed.

404
Not found.

408
Request timeout.

429
Too many requests (rate limited).

500
Internal server error.

502
Bad gateway: upstream sent a bad reply.

503
Service unavailable (overloaded or down).

504
Gateway timeout: upstream too slow.

SENPAI SAYS
2xx fine, 3xx go elsewhere, 4xx your fault, 5xx the server's fault.

28 CIDR & SUBNETS

The map room

/32
1 address (a single host).

/30
4 addresses.

/28
16 addresses (AWS: 11 usable).

/27
32 addresses.

/24
256 addresses (254 usable).

/22
1,024 addresses.

/20
4,096 addresses.

/16
65,536 addresses.

/8
16,777,216 addresses.

10.0.0.0/8
private range (large).

172.16.0.0/12
private range (medium).

192.168.0.0/16
private range (home/office).

127.0.0.1
localhost (loopback).

169.254.0.0/16
link-local, cloud metadata.

SENPAI SAYS
Formula: addresses = 2^(32 - prefix). Subtract 5 for AWS usable hosts.

29 INTERVIEW HOT QUESTIONS
Boss-fight Q&A

What is an inode?
Metadata for a file: owner, perms, data blocks. Not the name.

Hard link vs soft link?
Hard: another name for the same inode. Soft: a path pointer that can dangle.

What is a zombie process?
A finished child whose parent never called wait(). Fix the parent.

What is load average?
Runnable plus waiting tasks over 1, 5, 15 min. Compare with CPU count.

SIGTERM vs SIGKILL?
15 asks politely and can be handled. 9 is forced and cannot be caught.

passwd vs shadow?
/etc/passwd: user info. /etc/shadow: password hashes, root-only.

Boot process in brief?
Firmware (UEFI) > bootloader (GRUB) > kernel > systemd > services.

What happens when you type ls?
Shell parses, searches PATH, forks, execs, prints output.

TCP vs UDP?
TCP: reliable, ordered, handshake. UDP: fast, no guarantees.

Container vs VM?
Containers share the host kernel. VMs run their own kernel.

Pod vs Deployment?
Pod: smallest unit. Deployment: manages replicas and rollouts.

What is idempotency?
Running it twice gives the same result (Ansible, Terraform).

Blue/green vs canary?
Blue/green swaps whole environments. Canary shifts a small % first.

What is IaC?
Defining infrastructure in version-controlled code, not by clicking.

SENPAI SAYS
Explain it in one sentence, then give a real example. That's what impresses.

30 30-DAY QUEST MAP
Your roadmap

Week 1
Navigation, files, permissions, grep, pipes (Vol. 2 basics).

Week 2
Processes, systemd, logs, disks, networking, SSH keys.

Week 3
Bash scripting, cron, Git, Docker and Compose.

Week 4
Kubernetes, Terraform, Ansible, CI/CD and one cloud.

Final boss
Deploy a small app: Dockerfile > CI > Terraform > Kubernetes.

OverTheWire Bandit
a wargame that teaches Linux by playing.

Linux Journey
friendly step-by-step Linux lessons.

Killercoda
free browser labs for Linux and Kubernetes.

Play with Docker
try Docker in your browser.

SENPAI SAYS
Break a test VM on purpose, then fix it. Fastest way to really learn.

